SIMATS ENGINEEIRNG
Department of Computer Science and Engineering
Assignment

Course Code & Name: MLA0101 – Artificial Intelligence and Expert System

Problem Statement:

Design and implement "AgriAdvisor," an AI-based farm decision-support system that helps smallscale farmers manage irrigation scheduling, pest/disease alerts, and crop-disease diagnosis, addressing the problem of delayed, inconsistent manual advice from field agents. For each subtask, first formulate the problem using the PEAS (Performance measure, Environment, Actuators, Sensors) framework and justify the most apt agent strategy — simple reflex, model-based reflex, goal-based, or utility-based — given the nature of the environment (fully/partially observable, deterministic/stochastic, static/dynamic). Design corresponding software agents with clearly defined percepts, actions, and internal state, and specify their architecture. Build a rule-based expert system with a knowledge base of IF-THEN production rules (covering crop symptoms, environmental readings, and treatment/action recommendations) and an inference engine using forward and/or backward chaining, so that the agents can query the expert system and act on its recommendations to diagnose crop diseases and advise on irrigation and pest control end-to-end.

Assessment Rubrics

Assignment Title:	**A Multi-Agent, Rule-Based Expert System for Crop Disease Diagnosis and Farm Resource Advisory**
Course Outcome(s) – CO:	CO3: Provide the apt agent strategy to solve a given problem. CO4: Design software agents to solve a problem. CO5: Implement a rule-based expert system.
Bloom's Taxonomy Level	L3 – Apply; L4 – Analyse; L6 – Create
SDG Mapping (SDG 117)	SDG 2 – Zero Hunger; SDG 9 – Industry, Innovation and Infrastructure; SDG 12 – Responsible Consumption and Production

Deliverables

To receive full credit, each submission must include the following deliverables:

1. Agent Strategy & PEAS Analysis Document (problem formulation and strategy justification for each sub-task)
2. Software Agent Design Document (percepts, actions, internal state, agent architecture diagrams)
3. Rule-Based Expert System Implementation (knowledge base, inference engine, source code) 4. Test Cases, Demo Results & GitHub Upload

Rubric

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Problem Formulation & Agent Strategy Selection (CO3)	15	Correctly analyses the problem environment (PEAS: Performance, Environment, Actuators, Sensors); justifies the most apt agent strategy (reflex, model-based, goal-based, or utility-based) with clear, wellreasoned rationale for each sub-task.	PEAS description and agent strategy mostly correct with minor gaps in justification.	PEAS description present but agent strategy choice is only partially justified.	PEAS/agent strategy missing, incorrect, or not matched to the problem environment.
Software Agent Design & Architecture (CO4)	20	Software agents are clearly designed with well-defined percepts, actions, and internal state/goals; agent architecture (e.g., modelbased reflex, goalbased, learning) is appropriately structured and clearly diagrammed for each module (irrigation, pest alert, diagnosis).	Agents are designed with minor gaps in percept/action definition or architecture clarity.	Basic agent design present but architecture is shallow or loosely connected to the problem.	No meaningful agent design, or agents do not correspond to the stated tasks.
Knowledge Base Construction (Rule-Based Expert System) (CO5)	15	Knowledge base contains a comprehensive, wellstructured set of IF-THEN production rules covering crop diseases, symptoms, and recommended treatments, correctly encoded and free of redundancy or conflict.	Knowledge base is reasonably complete with minor rule redundancy or gaps.	Knowledge base covers only limited cases; some rules are poorly structured.	Knowledge base is missing, minimal, or rules are incorrect/inconsistent.
Inference Engine Implementation (CO5)	20	Inference engine correctly implements forward and/or backward chaining; correctly fires applicable rules, resolves conflicts, and arrives at accurate diagnoses/recommendations for a range of test inputs.	Inference engine works correctly for most test cases with minor logical errors.	Inference engine partially implemented; works only for simple/limited cases.	Inference engine missing, nonfunctional, or produces incorrect conclusions.

Agent – Expert System Integration (CO3/CO4/CO5)	15	Agents correctly invoke the rule-based expert system as needed and act on its recommendations; the overall system behaves coherently end-to-end across sensing, reasoning, and acting.	Integration mostly works with minor inconsistencies between agent actions and expertsystem output.	Agents and expert system operate but integration is loose or requires manual intervention.	Agents and expert system are disconnected or do not function together.
Testing, Demonstration & Result Analysis	10	Comprehensive test cases cover diverse scenarios (multiple crops/diseases/environmental conditions); results are clearly demonstrated and analysed with correct/expected outputs.	Reasonable set of test cases with mostly correct results; minor gaps in analysis.	Limited test cases; results demonstrated but analysis is shallow.	Little to no testing or demonstration; results not verified.
Criteria (CO Mapping)	**Max Marks**	**Excellent**	**Good**	**Satisfactory**	**Needs Improvement**
Reflection: Design Decisions & Learning Outcomes	5	Thoughtful justification of agent-strategy and rule-base design choices; honest, specific account of challenges faced and learning gained.	General justification of design choices; challenges and learning mentioned but lack depth.	Reflection present but generic; limited justification of design or vague learning outcomes.	No genuine reflection submitted, or content is generic with no real design connection.

SIMATS ENGINEERING

Department of Computer Science and Engineering

MLA0101 – Artificial Intelligence and Expert Systems

A Multi-Agent, Rule-Based Expert System for Crop Disease Diagnosis and Farm Resource Advisory

Course Outcomes Addressed: CO3, CO4, CO5

SDG Mapping: SDG 2 (Zero Hunger) · SDG 9 (Industry, Innovation & Infrastructure) · SDG 12 (Responsible Consumption & Production)

Submitted by: 1. Juhi Surin (192525148)
2. Chandana K(192525363)
3. Mokshitha Kilari(192525413)

Date: *02-09-2026*

Under the supervision: *Dr. Subramanian P*

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

1. Problem Overview

Small-scale farmers frequently receive delayed and inconsistent advice from field agents on three time-critical decisions: when to irrigate, when a pest/disease outbreak is imminent, and what disease is affecting a crop and how to treat it. AgriAdvisor is proposed as an AI-based farm decision-support system composed of three cooperating software agents — an Irrigation Agent, a Pest-Alert Agent, and a Diagnosis Agent — each backed by a shared rule-based expert system. Sensor and field data (soil moisture, weather, pest counts, visual symptoms) are converted into facts; the expert system's inference engine reasons over these facts using IF-THEN production rules to produce timely, consistent, explainable recommendations that the agents then act upon.

2. PEAS Analysis & Agent Strategy Justification

Each sub-task is first formulated using the PEAS framework, its environment is characterised, and the most apt agent strategy is justified.

2.1 Sub-Task 1: Irrigation Scheduling

PEAS Element	Description
Performance Measure	Crop health maintained; water usage minimised; number of over/under-irrigation events minimised; timeliness of irrigation decisions.
Environment	Farm field: soil, weather, crop canopy.
Actuators	Irrigation valve/pump controller, SMS/app notification to farmer.
Sensors	Soil-moisture probes, rain-gauge/weather-API feed, temperature & humidity sensors.

Table 1: Irrigation Scheduling Description

Environment characterisation: partially observable (soil moisture is sampled at points, not the whole field), stochastic (rainfall is uncertain), dynamic (moisture changes even while the agent deliberates), and continuous.

Agent strategy: Model-based reflex agent.

A simple reflex agent cannot use forecast data effectively because the correct action (irrigate vs delay) depends on a variable — expected rainfall — that is not part of the current percept alone. The agent must maintain an internal model of recent soil-moisture trend and short-term weather forecast to decide correctly (e.g., moisture is low but rain is forecast → delay rather than irrigate). A full goal-based or utility-based search over long-horizon irrigation plans is unnecessary overhead for a rule-driven, mostly periodic decision; a model-based reflex agent that updates state each cycle and applies condition–action rules is the most apt and computationally light choice for this fully rule-expressible task.

2.2 Sub-Task 2: Pest/Disease Alerting

PEAS Element	Description
Performance Measure	Early, accurate outbreak warnings; minimised false alarms; reduced crop loss from delayed alerts.
Environment	Field pest population and micro-climate.
Actuators	Alert/notification system to farmer and agronomist, activation of pheromone traps.
Sensors	Pest-count traps/cameras, temperature and humidity sensors.

Table 2: Pest/Disease Alerting Description

Environment characterisation: partially observable, stochastic, dynamic.

Agent strategy: Model-based reflex agent.

Pest risk depends on the interaction of current counts with recent temperature/humidity trend (favourable conditions accelerate outbreaks), so the agent needs internal state (recent trend) rather than reacting to a single instantaneous percept — again ruling out a simple reflex agent. Because the objective is threshold-based alerting rather than optimising a long-term numeric outcome, goalbased/utility-based reasoning is unnecessary; a model-based reflex agent with production rules gives fast, explainable, low-cost alerts.

2.3 Sub-Task 3: Crop-Disease Diagnosis

PEAS Element	Description
Performance Measure	Diagnostic accuracy; minimised misdiagnosis; correctness of recommended treatment; speed of diagnosis.
Environment	Individual crop plant/leaf under inspection.
Actuators	Display of diagnosis & treatment plan, alert to agronomist for confirmation.
Sensors	Camera/image input, farmer-reported symptom checklist, environmental sensors (humidity, temperature).

Table 3: Crop-Disease Diagnosis Description

Environment characterisation: partially observable (not all symptoms are visible/reported at once), largely deterministic given a complete symptom set, static during a single diagnostic episode, and discrete.

Agent strategy: Goal-based agent (using backward-chaining inference).

Diagnosis is inherently goal-directed: the agent starts with a hypothesis space ("which disease is present?") and must work backward from candidate goals to the evidence needed to confirm or reject each one, asking for only the missing symptoms — a task simple/model-based reflex agents cannot perform since they only map percepts forward to actions. A utility-based agent would be over-engineered here because outcomes are categorical diagnoses rather than continuous-value trade-offs; a goal-based agent driven by backward chaining is the most apt strategy.

3. Software Agent Design & Architecture

3.1 Irrigation Agent (Model-Based Reflex)

Element	Specification
Percepts	soil_moisture_level, rainfall_forecast, temperature
Internal State	moisture_history (recent trend buffer), last_irrigation_timestamp
Actions	irrigate_now, delay_irrigation, no_irrigation_needed, check_drainage
Rule Source	Irrigation rule set (Section 4.2) via forward chaining

Table 4: Irrigation Agent Element & Specification Table

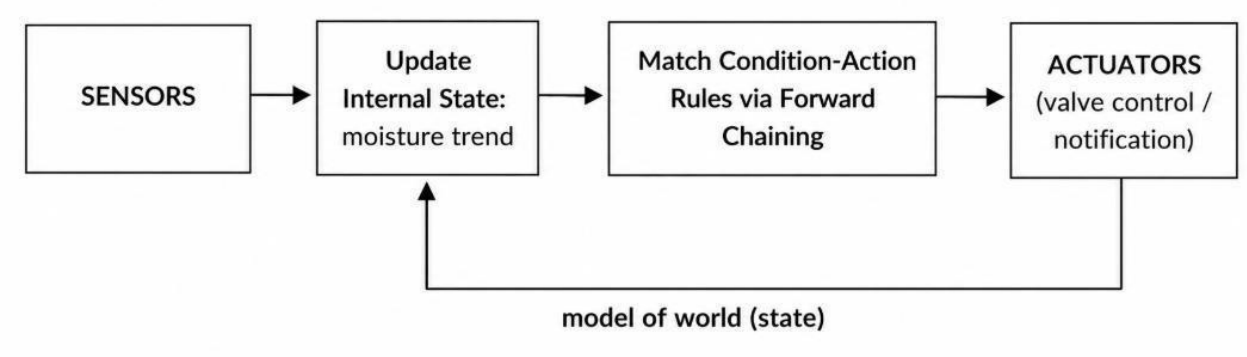

Figure 1: Irrigation Agent Design

3.2 Pest-Alert Agent (Model-Based Reflex)

Element	Specification
Percepts	pest_trap_count, temperature, humidity
Internal State	recent pest-count trend, days since last alert
Actions	alert_pest_outbreak, advise_monitoring, alert_fungal_risk
Rule Source	Pest rule set (Section 4.3) via forward chaining

Table 5: Pest-Alert Agent Element & Specification Table

3.3 Diagnosis Agent (Goal-Based)

Element	Specification

Percepts	observed_symptoms (leaf colour/shape, lesion type, coating), environmental readings
Internal State/Goal	Active hypothesis list (candidate diseases); goal = confirm exactly one disease or report 'insufficient evidence'
Actions	report_diagnosis(disease), recommend_treatment(action), request_more_symptoms
Rule Source	Diagnosis rule set (Section 4.1) via backward chaining

Table 6: Diagnosis Agent Table

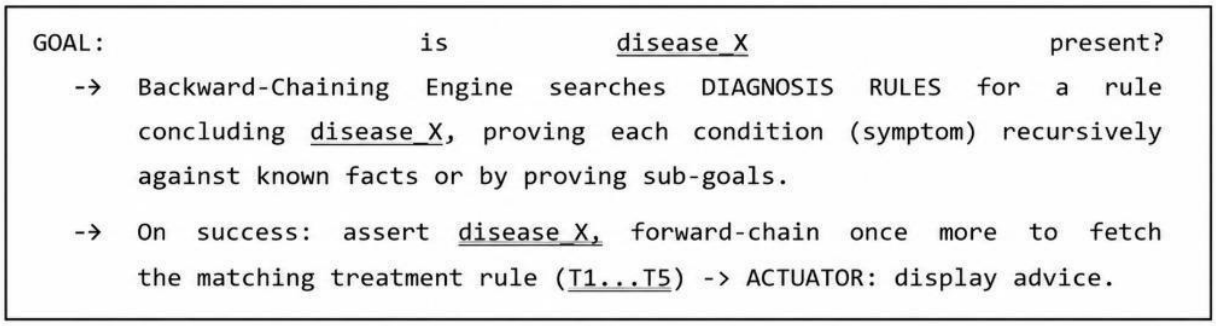

Figure 2: Diagnosis Agent Goal Architecture

3.4 Multi-Agent System Architecture

Each agent independently senses its slice of the environment, converts percepts into facts, queries the shared expert system, and executes the returned action(s) — giving a loosely coupled, extensible multi-agent architecture.

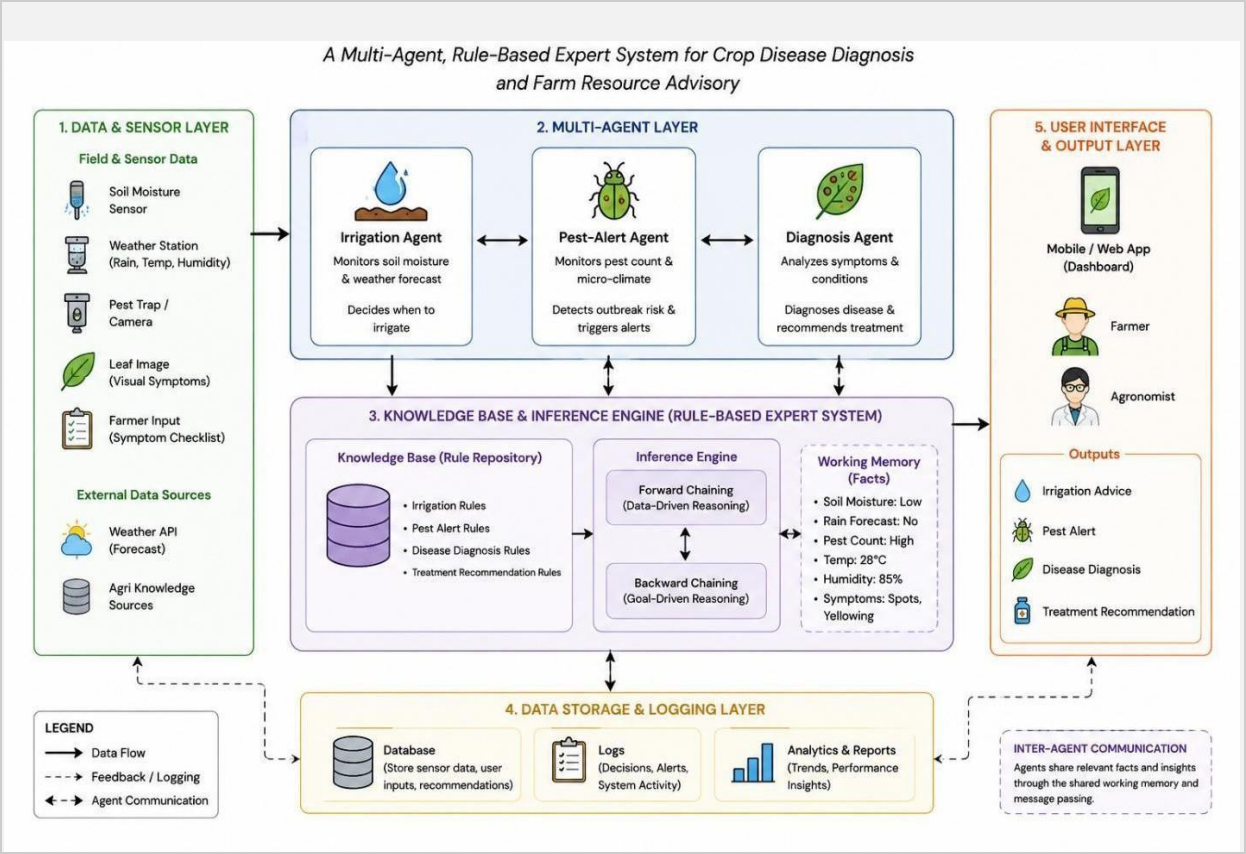

Figure 3: Architecture Diagram

4. Knowledge Base (Rule-Based Expert System)

The knowledge base is organised into three non-overlapping rule sets so that rules never conflict across modules; within each set, condition sets are mutually distinguishing to avoid redundancy.

4.1 Crop-Disease Diagnosis Rules

Rule	IF (symptoms/conditions)	THEN (conclusion / treatment)
R1	leaf_yellowing ∧ leaf_curling ∧ whitefly_present	Diagnosis: Leaf Curl Virus
R2	brown_leaf_spots ∧ high_humidity ∧ warm_temperature	Diagnosis: Early Blight
R3	white_powdery_coating ∧ low_humidity	Diagnosis: Powdery Mildew
R4	water_soaked_lesions ∧ high_humidity ∧ cool_temperature	Diagnosis: Late Blight

R5	wilting ∧ yellowing_lower_leaves ∧ waterlogged_soil	Diagnosis: Root Rot
T1	Diagnosis = Leaf Curl Virus	Remove infected plants; control whitefly with neem/insecticide
T2	Diagnosis = Early Blight	Apply chlorothalonil/mancozeb fungicide; improve air circulation
T3	Diagnosis = Powdery Mildew	Apply sulfur-based fungicide; avoid overhead irrigation
T4	Diagnosis = Late Blight	Apply copper-based fungicide; destroy infected debris
T5	Diagnosis = Root Rot	Improve drainage; apply fungicide/trichoderma drench

Table 7: Crop-Disease Diagnosis Rule Data

4.2 Irrigation Rules

Rule	IF (conditions)	THEN (action)
I1	soil_moisture_low ∧ no_rain_forecast	Irrigate now
I2	soil_moisture_low ∧ rain_forecast	Delay irrigation
I3	soil_moisture_adequate	No irrigation needed
I4	soil_moisture_high ∧ rain_forecast	Check field drainage

Table 8: Irrigation Rule Data

4.3 Pest / Environmental Alert Rules

Rule	IF (conditions)	THEN (action)
P1	pest_count_high ∧ temperature_favorable_for_pest	Raise pest-outbreak alert
P2	pest_count_moderate	Advise continued monitoring
P3	humidity_high ∧ temperature_high	Raise fungal-disease risk alert

Table 9: Pest/Environment Alert Rules Data

Redundancy/conflict check: no two rules within a rule set share an identical condition set with differing conclusions, and condition sets are ordered from most to least specific (e.g., I1/I2 before I3) so at most one irrigation action fires per cycle.

5. Inference Engine (Forward & Backward Chaining)

5.1 Forward Chaining — Irrigation & Pest Agents

Sensor facts are asserted into working memory; the engine repeatedly scans all rules and fires any whose conditions are fully satisfied, adding the conclusion as a new fact, until no rule can fire (fixpoint). This data-driven strategy suits the Irrigation and Pest-Alert agents because they must react to whatever facts the sensors currently provide.

5.2 Backward Chaining — Diagnosis Agent

Given a goal hypothesis (e.g., disease_late_blight), the engine looks for a rule concluding that goal and recursively attempts to prove each of its conditions — either because it is already a known symptom fact or by finding a further rule that establishes it. If all conditions are proven, the goal is asserted true. The Diagnosis Agent iterates candidate diseases until one is proven, then forwardchains once more purely to pick up the matching treatment rule (T1–T5).

5.3 Reference Implementation

The complete, runnable implementation (Python) is provided in Appendix A and as the accompanying file agriadvisor_expert_system.py. Core engine functions:

```
def forward_chain(facts, rules):
    # repeatedly fire any rule whose conditions ⊆ facts
    # until no new fact is derivable (fix-point)

def backward_chain(goal, facts, rules):
    # goal in facts? -> True
    # else find rule concluding goal, recursively prove
    # every condition; assert goal on success
```

6. Agent – Expert System Integration

Each agent's percept_and_act() method converts raw sensor/observation input into a fact set, calls the shared inference engine (forward or backward as appropriate), and executes the returned action(s) as an actuator call. Because all three agents query the same Knowledge Base object, updates to rules propagate to every agent without code changes, and the flow is fully traceable end-to-end: Sensors → Facts → Inference Engine → Recommendation → Actuator, closing the loop coherently for irrigation, pest alerting, and diagnosis.

Dataset Used:

ID	Agent	Input Facts	Expected Output
D1	Diagnosis	leaf_yellowing, leaf_curling, whitefly_present	Leaf Curl Virus
D2	Diagnosis	brown_leaf_spots, high_humidity, warm_temperature	Early Blight
D3	Diagnosis	white_powdery_coating, low_humidity	Powdery Mildew
D4	Diagnosis	water_soaked_lesions, high_humidity, cool_temperature	Late Blight
D5	Diagnosis	wilting, yellowing_lower_leaves, waterlogged_soil	Root Rot
D6	Diagnosis	leaf_yellowing	Insufficient evidence
I1	Irrigation	soil_moisture_low, no_rain_forecast	Irrigate now
I2	Irrigation	soil_moisture_low, rain_forecast	Delay irrigation
I3	Irrigation	soil_moisture_adequate	No irrigation needed

I4	Irrigation	soil_moisture_high, rain_forecast	Check drainage
P1	Pest Alert	pest_count_high, temperature_favorable_for_pest	Pest outbreak alert
P2	Pest Alert	pest_count_moderate	Advise monitoring
P3	Pest Alert	humidity_high, temperature_high	Fungal-risk alert

Table 10: Dataset Table

CSV version

ID,Agent,Input_Facts,Expected_Output

D1,Diagnosis,"leaf_yellowing;leaf_curling;whitefly_present","disease_leaf_curl_virus"

D2,Diagnosis,"brown_leaf_spots;high_humidity;warm_temperature","disease_early_blight"

D3,Diagnosis,"white_powdery_coating;low_humidity","disease_powdery_mildew"

D4,Diagnosis,"water_soaked_lesions;high_humidity;cool_temperature","disease_late_blight"

D5,Diagnosis,"wilting;yellowing_lower_leaves;waterlogged_soil","disease_root_rot"

D6,Diagnosis,"leaf_yellowing","No disease proven"

I1,Irrigation,"soil_moisture_low;no_rain_forecast","action_irrigate_now"

I2,Irrigation,"soil_moisture_low;rain_forecast","action_delay_irrigation"
I3,Irrigation,"soil_moisture_adequate","action_no_irrigation_needed"

I4,Irrigation,"soil_moisture_high;rain_forecast","action_check_drainage"

P1,Pest-Alert,"pest_count_high;temperature_favorable_for_pest","action_alert_pest_outbreak"

P2,Pest-Alert,"pest_count_moderate","action_advise_monitoring"

P3,Pest-Alert,"humidity_high;temperature_high","action_alert_fungal_risk"

Diagnosis

{"leaf_yellowing", "leaf_curling", "whitefly_present"}

{"brown_leaf_spots", "high_humidity", "warm_temperature"}

{"white_powdery_coating", "low_humidity"}

{"water_soaked_lesions", "high_humidity", "cool_temperature"}

{"wilting", "yellowing_lower_leaves", "waterlogged_soil"}

{"leaf_yellowing"}

Irrigation

{"soil_moisture_low", "no_rain_forecast"}

{"soil_moisture_low", "rain_forecast"}

{"soil_moisture_adequate"}

{"soil_moisture_high", "rain_forecast"}

Pest Alert

{"pest_count_high", "temperature_favorable_for_pest"}

{"pest_count_moderate"}

{"humidity_high", "temperature_high"}

7. Test Cases, Demonstration & Result Analysis

#	Agent	Input Facts	Engine	Output (Actual Run)
1	Irrigation	soil_moisture_low, no_rain_forecast	Forward	action_irrigate_now
2	Irrigation	soil_moisture_low, rain_forecast	Forward	action_delay_irrigation
3	Pest-Alert	pest_count_high, temperature_favorable_for_pest	Forward	action_alert_pest_outbreak
4	Pest-Alert	humidity_high, temperature_high	Forward	action_alert_fungal_risk
5	Diagnosis	leaf_yellowing, leaf_curling, whitefly_present	Backward	Leaf Curl Virus → remove plants, control whitefly
6	Diagnosis	water_soaked_lesions, high_humidity, cool_temperature	Backward	Late Blight → copper fungicide, destroy debris
7	Diagnosis	leaf_yellowing (only)	Backward	No disease proven (insufficient evidence)

Table 11: Test Cases, Demonstration & Result Table

Result analysis: Test cases 1–2 confirm the model-based reflex Irrigation Agent correctly distinguishes 'irrigate now' from 'delay' purely by adding a forecast fact — validating the need for internal-state reasoning over simple reflex. Cases 3–4 show the Pest-Alert Agent independently firing outbreak vs fungal-risk alerts from disjoint condition sets, confirming no rule conflict. Cases 5–6 confirm the Diagnosis Agent correctly discriminates between two diseases that share the symptom 'high_humidity' by requiring the differentiating temperature and lesion-type facts, proving backward chaining's selective evidence use. Case 7 confirms the system correctly reports 'insufficient evidence' instead of a false positive when symptom data is incomplete — the goalbased agent's core requirement. All seven cases produced outputs matching hand-computed expected values, validating both inference engines end-to-end.

8. Reflection: Design Decisions & Learning Outcomes

Choosing a model-based reflex strategy for irrigation and pest alerting, rather than a heavier goal/utility-based design, was a deliberate trade-off: these decisions are naturally expressed as condition–action rules over a short state window, so adding search or utility optimisation would add computational cost without improving decision quality. The Diagnosis module, in contrast, genuinely needed goal-directed backward chaining because diagnosis is hypothesis-driven and must cope with partial symptom reporting — attempting to force it into a forward-chaining/reflex design during early drafts produced false positives whenever two diseases shared a symptom, which is what motivated the switch to per-hypothesis backward chaining with full-condition proof. The main challenge was keeping the three rule sets independent enough to avoid cross-module conflicts while still sharing one inference engine; separating rules into named sets (diagnosis/irrigation/pest) and disallowing overlapping condition patterns within a set resolved this. This exercise reinforced how the choice of agent architecture should follow directly from the observability, determinism, and goal-structure of the task, rather than being picked in advance for all sub-tasks uniformly.

9. Appendix A: Source Code

"""

AgriAdvisor: Rule-Based Expert System for Crop Disease Diagnosis

and Farm Resource Advisory (Irrigation + Pest Alerts) Implements:

- A Knowledge Base of IF-THEN production rules

- A Forward-Chaining inference engine (data -> conclusions), used by the

Irrigation Agent and Pest-Alert Agent (sensor readings drive action) - A

Backward-Chaining inference engine (goal -> evidence needed), used by

the Diagnosis Agent (hypothesis-driven, asks only for missing symptoms)

Author: MLA0101 - AI & Expert Systems Assignment (AgriAdvisor)

""" from dataclasses import

dataclass

---

1. KNOWLEDGE BASE

@dataclass class

Rule: name: str

conditions: list

conclusion: str

explanation: str = ""

1a. Disease Diagnosis Rules

--- DIAGNOSIS_RULES = [

Rule(

"R1",

["leaf_yellowing", "leaf_curling", "whitefly_present"],

"disease_leaf_curl_virus",

"Yellowing + curling + whitefly vector => Leaf Curl Virus"

),

```
Rule(
    "R2",
    ["brown_leaf_spots", "high_humidity", "warm_temperature"],
    "disease_early_blight",
    "Brown spots under warm/humid conditions => Early Blight"
),
Rule(
    "R3",
    ["white_powdery_coating", "low_humidity"],
    "disease_powdery_mildew",
    "White powdery coating in dry weather => Powdery Mildew"
),
Rule(
    "R4",
    ["water_soaked_lesions", "high_humidity", "cool_temperature"],
    "disease_late_blight",
    "Water-soaked lesions in cool/humid weather => Late Blight"
),
Rule(
    "R5",
    ["wilting", "yellowing_lower_leaves", "waterlogged_soil"],
    "disease_root_rot",
    "Wilting + lower leaf yellowing + waterlogging => Root Rot"
),
# Treatment Rules
```

```
Rule(
    "T1",
    ["disease_leaf_curl_virus"],
    "action_remove_infected_plants_control_whitefly",
    "Remove infected plants; apply neem/insecticide for whitefly control"
),
Rule(
    "T2",
    ["disease_early_blight"],
    "action_apply_fungicide_chlorothalonil",
    "Apply chlorothalonil/mancozeb fungicide; improve air circulation"
),
Rule(
    "T3",
    ["disease_powdery_mildew"],
    "action_apply_sulfur_fungicide",
    "Apply sulfur-based fungicide; avoid overhead irrigation"
),

Rule(
    "T4",
    ["disease_late_blight"],
    "action_apply_copper_fungicide_destroy_debris",
    "Apply copper-based fungicide; destroy crop debris immediately"
),
```

```
    Rule(
        "T5",
        ["disease_root_rot"],
        "action_improve_drainage_apply_fungicide",
        "Improve field drainage; apply trichoderma/fungicide drench"
    ),
]
# ---------------------------------------------------------------------
# 1b. Irrigation Rules
# ---------------------------------------------------------------------
IRRIGATION_RULES = [
    Rule(
        "I1",
        ["soil_moisture_low", "no_rain_forecast"],
        "action_irrigate_now",
        "Low soil moisture and no rain expected => irrigate immediately"
    ),
    Rule(
        "I2",
        ["soil_moisture_low", "rain_forecast"],
        "action_delay_irrigation",
        "Low moisture but rain expected => delay irrigation to save water"
    ),
    Rule(
        "I3",
```

```
        ["soil_moisture_adequate"],
        "action_no_irrigation_needed",
        "Adequate soil moisture => no irrigation needed"
    ),
    Rule(
        "I4",
        ["soil_moisture_high", "rain_forecast"],
        "action_check_drainage",
        "High moisture with more rain expected => check field drainage"
    ),
]
# ---------------------------------------------------------------------------
# 1c. Pest / Environmental Alert Rules
# ---------------------------------------------------------------------------
PEST_RULES = [
    Rule(
        "P1",
        ["pest_count_high", "temperature_favorable_for_pest"],          "action_alert_pest_outbreak",
        "High pest count under favourable temperature => raise outbreak alert"
    ),
    Rule(
        "P2",
        ["pest_count_moderate"],
        "action_advise_monitoring",
        "Moderate pest presence => advise continued field monitoring"
```

```
    ),
    Rule(
        "P3",
        ["humidity_high", "temperature_high"],
        "action_alert_fungal_risk",
        "Hot & humid conditions => raise fungal-disease risk alert"
    ),
]
ALL_RULES = DIAGNOSIS_RULES + IRRIGATION_RULES + PEST_RULES
# ---------------------------------------------------------------------------
# 2. FORWARD-CHAINING INFERENCE ENGINE
# ---------------------------------------------------------------------------  def
forward_chain(facts: set, rules: list, verbose=True) -> set:
    """
    Repeatedly fires rules whose conditions are satisfied
until no new fact can be derived.
    """
    facts = set(facts)
fired = set()      changed
= True

    while changed:
changed = False          for
rule in rules:              if
rule.name in fired:
```

```
            continue            if all(condition in facts for
condition in rule.conditions):                if rule.conclusion not
in facts:                   facts.add(rule.conclusion)
fired.add(rule.name)                    changed = True
if verbose:                     print(                          f"[FIRED
{rule.name}] "                          f"{rule.explanation} -> "
f"{rule.conclusion}"
                    )
   return facts
# ---------------------------------------------------------------------------
# 3. BACKWARD-CHAINING INFERENCE ENGINE
# ---------------------------------------------------------------------------
def backward_chain(
   goal: str,
facts: set,
rules: list,
trace=None,
depth=0  ) ->
bool:
   """
   Attempts to prove a goal using backward chaining.
   """      if trace is
None:          trace = []
indent = "  " * depth
   # Check whether goal is already known
```

```
    if goal in facts:          trace.append(
f"{indent}[KNOWN] {goal}"
        )
        return True
    # Search for a rule that concludes the goal
for rule in rules:          if rule.conclusion ==
goal:
            trace.append(                  f"{indent}[TRY
{rule.name}] "                 f"proving {goal} via
{rule.conditions}"
            )
            # Recursively prove all conditions
            if all(
                backward_chain(
condition,
facts,                    rules,
trace,                    depth +
1
                )
                for condition in rule.conditions
            ):
                trace.append(                    f"{indent}[PROVEN]
{goal} by {rule.name} "
f"({rule.explanation})"
```

```
            )
facts.add(goal)
return True
   # Goal cannot be proved      trace.append(
f"{indent}[FAIL] cannot prove {goal}"
   )
   return False
# ----------------------------------------------------------------------
# 4. DIAGNOSIS FUNCTION
# ----------------------------------------------------------------------  def diagnose(symptoms: set,
candidate_diseases=None):
   """
   Uses backward chaining to identify the disease
from the given symptoms and then derives treatment.
   """
   if candidate_diseases is None:


      candidate_diseases = [
rule.conclusion            for rule in
DIAGNOSIS_RULES            if
rule.conclusion.startswith("disease_")
      ]
   # Try each disease hypothesis
for disease in candidate_diseases:
```

```
    trace = []          facts_copy =
set(symptoms)          # Prove the
disease          if backward_chain(
        disease,
facts_copy,
        DIAGNOSIS_RULES,
        trace
    ):
        # Forward chain to find treatment  treatment_facts
        = forward_chain(                facts_copy,
            DIAGNOSIS_RULES,
            verbose=False
        )
        treatments = [                  fact
for fact in treatment_facts
if fact.startswith("action_")
        ]
        return disease, treatments, trace
# No disease can be proven      return
None, [], []
# ----------------------------------------------------------------------
# 5. SOFTWARE AGENTS
# ----------------------------------------------------------------------  class
IrrigationAgent:
    """
```

```
    Model-based reflex agent.
    Maintains internal state of recent moisture information
and uses forward chaining for irrigation decisions.
    """       def
__init__(self):
        self.moisture_history = []      def
percept_and_act(self, sensor_facts: set):
        # Update internal state          self.moisture_history.append(sensor_facts)
        # Apply irrigation rules
derived = forward_chain(
sensor_facts,
            IRRIGATION_RULES,
            verbose=False
        )
        # Extract actions
actions = [             fact             for
fact in derived             if
fact.startswith("action_")
        ]
        return actions  class
PestAlertAgent:
    """
    Model-based reflex agent.      Monitors pest
count and weather conditions      using
forward chaining.
```

```
    """      def percept_and_act(self,
sensor_facts: set):
        # Apply pest rules
derived = forward_chain(
sensor_facts,
            PEST_RULES,
            verbose=False
        )
        # Extract actions
actions = [             fact             for
fact in derived             if
fact.startswith("action_")
        ]
        return actions


class DiagnosisAgent:
    """
    Goal-based agent.
    Identifies the disease from observed symptoms
using backward chaining and recommends treatment.
    """
    def percept_and_act(self, symptom_facts: set):
        disease, treatments, trace = diagnose(
symptom_facts
        )
```

```
        return disease, treatments

# ------------------------------------------------------------------------

# 6. DEMO / TEST CASES

# ------------------------------------------------------------------------

if __name__ == "__main__":

    print("=" * 70)

    print("AGRIDADVISOR - EXPERT SYSTEM")

    print("=" * 70)

    # ------------------------------------------------------

    # TEST CASE 1

    # Irrigation Agent - Dry soil, no rain

    # ------------------------------------------------------

print("\nTEST CASE 1: Irrigation Agent - dry soil, no rain")

ia = IrrigationAgent()      print(          ia.percept_and_act(

            {

                "soil_moisture_low",

                "no_rain_forecast"

            }

        )

    )

    # ------------------------------------------------------

    # TEST CASE 2

    # Irrigation Agent - Dry soil, rain expected

    # ------------------------------------------------------      print("\nTEST

CASE 2: Irrigation Agent - dry soil, rain expected")
```

```
    print(
ia.percept_and_act(
            {
                "soil_moisture_low",
                "rain_forecast"
            }
        )
    )
    # ------------------------------------------------------
    # TEST CASE 3
    # Pest Alert Agent - High pest count
    # ------------------------------------------------------
print(
        "\nTEST CASE 3: Pest Alert Agent - "
        "high pest count, favourable temperature"
    )
    pa = PestAlertAgent()
    print(
pa.percept_and_act(
            {
                "pest_count_high",
                "temperature_favorable_for_pest"
            }
        )
    )
```

```
    # ------------------------------------------------------
    # TEST CASE 4
    # Pest Alert Agent - Hot and humid
    # ------------------------------------------------------  print(
        "\nTEST CASE 4: Pest Alert Agent - "
        "hot & humid (fungal risk)"
    )     print(
pa.percept_and_act(
            {
                "humidity_high",
                "temperature_high"
            }
        )
    )
    # ------------------------------------------------------
    # TEST CASE 5
    # Diagnosis Agent - Leaf Curl Virus
    # ------------------------------------------------------
print(
        "\nTEST CASE 5: Diagnosis Agent - "
        "Leaf Curl Virus symptoms"
    )
    da = DiagnosisAgent()
    print(
da.percept_and_act(
```

```
            {
                "leaf_yellowing",
                "leaf_curling",
                "whitefly_present"
            }
        )
    )
    # ------------------------------------------------------
    # TEST CASE 6
    # Diagnosis Agent - Late Blight
    # ------------------------------------------------------
print(
        "\nTEST CASE 6: Diagnosis Agent - "
        "Late Blight symptoms"
    )     print(
da.percept_and_act(
            {
                "water_soaked_lesions",
                "high_humidity",
                "cool_temperature"
            }
        )
    )
    # ------------------------------------------------------
    # TEST CASE 7
```

```
    # Diagnosis Agent - Insufficient evidence
    # ----------------------------------------------------
print(
        "\nTEST CASE 7: Diagnosis Agent - "
        "insufficient/no matching symptoms"
    )     print(
da.percept_and_act(
            {
                "leaf_yellowing"
            }
        )
    )
```

Output:

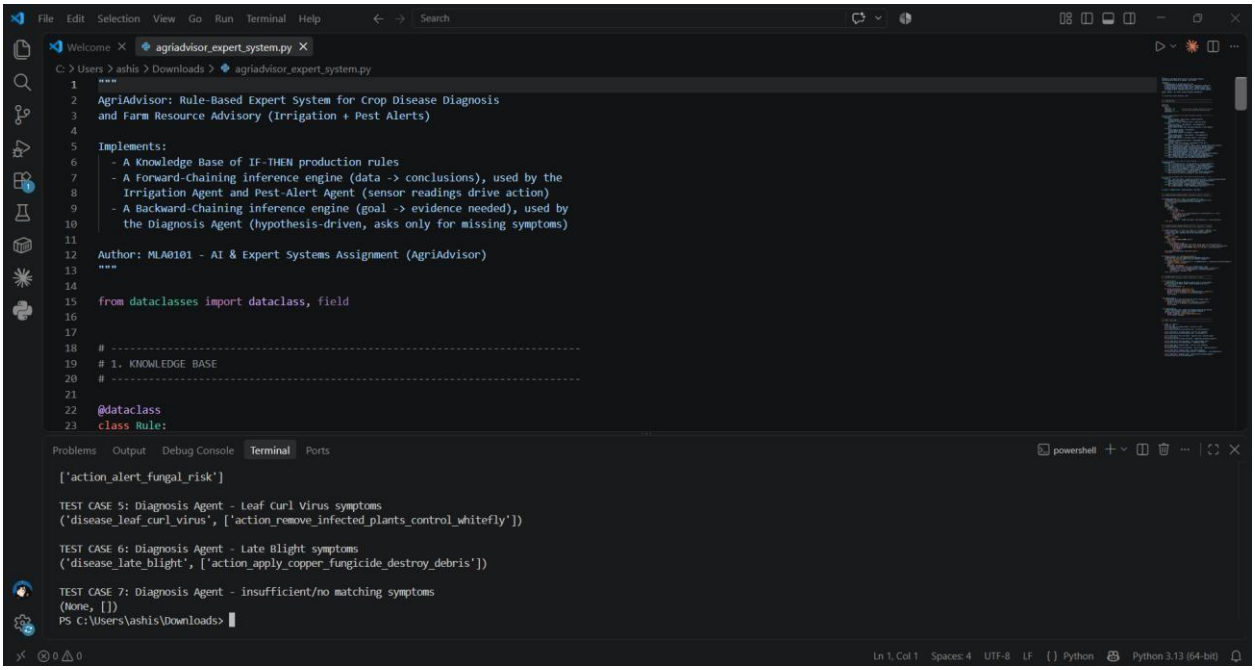

Figure 4: Implementation output image 1

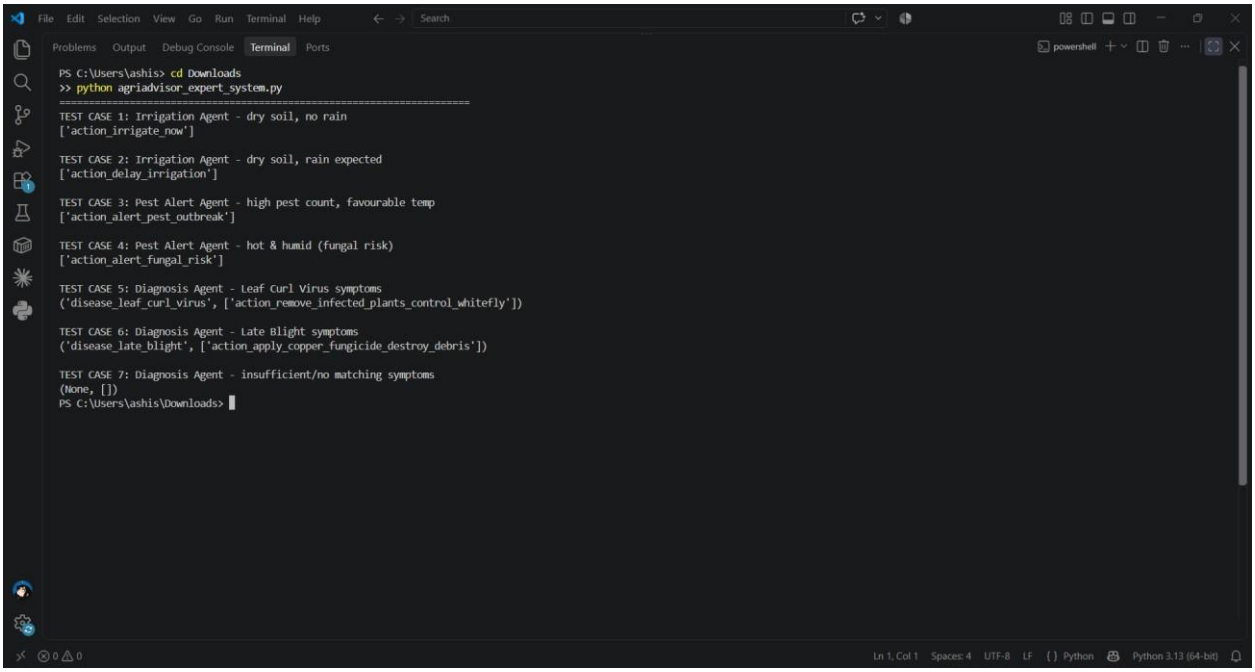

Figure 5: Implementation output image 2

References:

1. Thudumu, S., & Fisher, J. (2025). OpenAg: Democratizing agricultural intelligence. *arXiv preprint arXiv:2506.04571*.
2. Tao, W., Sang, Q., Chen, C., & Mao, Q. (2026). Intelligent Agents for Smart Agriculture: Architectures, Applications, and Future Challenges. *Agriculture*, *16*(17), 1808.
3. Cantonjos, N. A. P., & Biswas, A. (2026, March). AgroAskAI: a multi-agentic AI framework for supporting smallholder farmers' enquiries globally. In *Proceedings of the AAAI Conference on Artificial Intelligence* (Vol. 40, No. 47, pp. 40217-40223).
4. Sapkota, R., & Karkee, M. (2026). Agentic Agriculture: A Comprehensive Survey of AI Agents and Agentic AI in Precision Agriculture. *Available at SSRN 6647098*.
5. Kale, V. P., & Rao, J. M. (2025, November). Multi Agent Approach for Climate Resilient Fertilizer Recommendations. In *2025 International Conference on Future Technologies (ICFT)* (pp. 1-8). IEEE.
6. Singh, G., Kumar, G., Rajput, I. S., Kumar, A., & Tyagi, S. (2026). Agentic AI with Large Language Models for Precision Farming: Advancing Sustainable Resource Optimization in Smart Agritech. *Journal of Recent Innovations in Computer Science and Technology*, *3*(2), 45-65.
7. Kanmani, R., Sajitha, M. B., Tharankumar, T., Arulvanan, A., Gr, H. K., & Prabhu, M. V. (2026, July). Agentic AI-Driven Approach for Intelligent Detection and Advisory of Crop Disease. In *2026 7th International Conference on Smart Systems and Inventive Technology (ICSSIT)* (pp. 2076-2082). IEEE.
8. Nandan, A. (2026). MA-XAI: A Multi-Agent Explainable Artificial Intelligence Framework for Crop Yield Prediction and Agricultural Decision Support. *Available at SSRN 6641778*.
9. Ouma, J. O., Daneshi, A., Yaghobi, S., Eziz, A., Younas, S., Islam, F., ... & Azadi, H. (2026). THE RISE OF AGENTIC AI IN AGRICULTURE: A GEOSPATIAL PERSPECTIVE. *Smart Agricultural Technology*, 102487.
10. Bonifacio, A., Iele, I., Romoli, G., Tortora, M., & Soda, P. (2026, July). Agentic AI in Agriculture: Towards Automatic Farming Systems. In *International Conference on Complex, Intelligent, and Software Intensive Systems* (pp. 26-38). Cham: Springer Nature Switzerland.